
HavenWorld Build Guide

Part 9B — Launch Readiness (Part 2)

Self-Hosted Analytics • Player Onboarding Flow • Live Monitoring Dashboard • Seasonal Events • Patch Notes Modal

Author: Daniel **Date:** Friday, 18 September 2026 **Time:** 17:30 CDT **Location:** Anoka, MN

Series: Part 9 of 10 **License:** GPL-3.0 — Open Source, Free-Tier Only

Prerequisite: Part 9A Complete

HavenWorld Build Guide Series:

Parts 1

—

8 (Core stack, Auth, World Engine, Features, Economy, CI/CD, Sentry)

→

Part 9A (PWA, R2 CDN, Zero-Downtime Deploy, Beta Gate)

→

Part 9B [YOU ARE HERE]

(Analytics, Onboarding, Monitoring, Seasons, Patch Notes)

→

Part 10 (Scaling, Community, Roadmap)

Continuation Notice:

This document is a direct continuation of Part 9A. All infrastructure from Parts 1

—

9A must be operational before beginning Part 9B. The server is running on Oracle Cloud ARM VM via PM2 + Nginx; the client is deployed to

havenworld.pages.dev

via Cloudflare Pages.

Table of Contents

Section	Topic	Platform	Subsections
1	Self-Hosted Game Analytics Dashboard	MAC + LINUX	1.1 - 1.6
2	Player Onboarding Flow	MAC + LINUX	2.1 - 2.6
3	Live Monitoring Dashboard	LINUX MAC	3.1 - 3.6
4	Seasonal Event Activation System	MAC + LINUX	4.1 - 4.6
5	Patch Notes & "What's New" Modal	MAC + LINUX	5.1 - 5.6
6	Part 9B Completion Checklist	—	5 groups, 40+ items
7	What's Next — Part 10 Preview	—	—

Platform Tags Used Throughout This Document

MAC

Steps that run on your local macOS development machine (M-series or Intel).

LINUX

Steps that run on the Oracle Cloud ARM VM via SSH.

MAC + LINUX

Steps that apply to both environments.

§1 Self-Hosted Game Analytics Dashboard MAC + LINUX

1.1 — Why Build vs. Buy

Commercial analytics platforms such as Mixpanel, Amplitude, and Google Analytics 4 offer generous free tiers, but they introduce three problems that are unacceptable for HavenWorld's open-source, privacy-first audience:

- **Third-party data sharing:** These platforms aggregate user behavioral data across their entire customer base for model training and product development — a GDPR and COPPA liability risk for a game that may have younger players.
- **No game-domain awareness:** Commercial analytics understand pageviews and button clicks, not DAU, session length distributions, room popularity, economy health, or mini-game completion funnels.
- **Cookie consent friction:** Any third-party tracker requires a GDPR cookie banner on first load — adding friction before the player even sees Haven Park for the first time.

HavenWorld self-hosts analytics using the existing PostgreSQL instance already running on the Oracle VM. Zero new services. Zero new cost. Full data ownership.

Architecture Decision

Analytics events are written fire-and-forget to a
`GameEvent`
table. Dashboard queries run as raw SQL via Prisma's
`$queryRaw`
. A Chart.js frontend renders the data at
`/admin/analytics`
, accessible only to users with the
ADMIN
role.

1.2 — The GameEvent Prisma Model MAC

Open `packages/shared/prisma/schema.prisma` and add the following model after the existing models. Do not remove or alter any existing model definitions.

```
model GameEvent {
  id          Int      @id @default(autoincrement())
  userId      Int?      // null for anonymous events
  eventType   String    // LOGIN, LOGOUT, ROOM_JOIN,
                        // FISH_CAUGHT,
                        // FISH_THROWN, etc.
  eventData   Json?     // flexible
  payload     String    // roomId, itemId, coinAmount...
  sessionId   String?
  createdAt   DateTime  // UUID assigned at login, spans entire session
  @default(now()) @db.Timestamp(6) user      User?
  @relation(fields: [userId], references: [id], onDelete: SetNull)
  @@index([eventType, createdAt]) // fast queries for dashboards
  @@index([userId, createdAt])    // fast queries for per-user
  analytics   @@index([createdAt]) // time-range queries }
```

Also update the `User` model to reference the new relation and add session tracking fields:

```
model User { // ... existing fields ... gameEvents GameEvent[] lastLoginAt
DateTime? loginCount Int @default(0) }
```

Run the migration from your local machine (it will connect to the dev database using `DATABASE_URL` in your `.env`):

```
# [MAC] – run from the monorepo root npx prisma migrate dev --name
add_game_events
```

Tip — Three Indexes on GameEvent

The three

`@@index`

declarations are intentional. Dashboard queries filter on

`eventType + createdAt`

, per-user analytics filter on

`userId + createdAt`

, and time-range aggregations use

`createdAt`

alone. PostgreSQL will use the most selective index per query automatically.

1.3 — Analytics Service (Server) MAC + LINUX

Create the file `apps/server/src/services/analytics.ts`. This is the single entry point for all event tracking in the server codebase. The most important design constraint is that **analytics failures must never break gameplay** — every database write is wrapped in a try/catch that logs but does not re-throw.

```
import prisma from '../lib/prisma.js'; export type EventType = | 'LOGIN' |
'LOGOUT' | 'ROOM_JOIN' | 'ROOM_LEAVE' | 'CHAT_MESSAGE' | 'FISH_CAUGHT'
| 'PIZZA_COMPLETED' | 'CRAFTING_COMPLETED' | 'TRADE_COMPLETE' |
'TRADE_CANCELLED' | 'ITEM_PURCHASED' | 'ITEM_GIFTED' |
'AVATAR_CUSTOMIZED' | 'ACHIEVEMENT_UNLOCKED' | 'PET_ADOPTED' |
'TUTORIAL_STEP_COMPLETE' | 'TUTORIAL_FINISHED' | 'FRIEND_REQUEST_SENT' |
'FRIEND_ADDED' | 'REPORT_SUBMITTED' | 'SESSION_HEARTBEAT'; //
emitted every 5 min while player is online export interface EventPayload {
userId?: number; sessionId?: string; eventType: EventType; eventData?:
```

```
Record<string, unknown>; } // Fire-and-forget – analytics failures NEVER
break gameplay export async function trackEvent(payload: EventPayload):
Promise<void> { try { await prisma.gameEvent.create({ data: {
userId: payload.userId ?? null, sessionId: payload.sessionId ??
null, eventType: payload.eventType, eventData:
payload.eventData ?? {}, }, }); } catch (err) { // Log but
never throw – analytics is non-critical console.error('[Analytics] Failed
to track event:', err); } } // Session heartbeat – called from Socket.io
every 5 minutes per connected user export async function heartbeat(userId:
number, sessionId: string): Promise<void> { await trackEvent({ userId,
sessionId, eventType: 'SESSION_HEARTBEAT' }); }
```

1.4 — Integrate trackEvent into Game Systems MAC + LINUX

The following snippets show exactly where to call `trackEvent` within each existing server file. Add only the lines shown — do not replace surrounding logic.

In `apps/server/src/routes/auth.ts` — inside the POST `/auth/login` success handler, immediately after the JWT is issued:

```
import { trackEvent } from '../services/analytics.js'; // After successful
login – generate a sessionId and attach it to JWT payload: const sessionId =
crypto.randomUUID(); await trackEvent({ userId: user.id, sessionId,
eventType: 'LOGIN', eventData: { ip: req.ip }, }); // Return sessionId in
JWT payload so client passes it in Socket.io auth header
```

In `apps/server/src/socket/RoomManager.ts` — on room join and room leave:

```
// On ROOM_JOIN: trackEvent({ userId: socket.data.userId, sessionId:
socket.data.sessionId, eventType: 'ROOM_JOIN', eventData: { roomId }, });
// On ROOM_LEAVE: trackEvent({ userId: socket.data.userId, sessionId:
socket.data.sessionId, eventType: 'ROOM_LEAVE', eventData:
{ roomId }, });
```

In `apps/server/src/socket/handlers/fish.ts` — on fish caught:

```
trackEvent({ userId, sessionId, eventType: 'FISH_CAUGHT', eventData:
{ fishId: catchResult.fishId, weight: catchResult.weight, coins:
reward }, });
```

In `apps/server/src/socket/handlers/trade.ts` — on trade complete:

```
trackEvent({ userId, sessionId, eventType: 'TRADE_COMPLETE',
eventData: { partnerId, itemIds: offeredItems.map(i => i.id) }, });
```

In `apps/server/src/socket/handlers/economy.ts` — on item purchased:

```
trackEvent({ userId, sessionId, eventType: 'ITEM_PURCHASED',
eventData: { itemId, price, currency: 'COINS' }, });
```

In `apps/server/src/socket/index.ts` — add the 5-minute heartbeat inside the connection handler:

```
io.on('connection', (socket) => { // ... existing connection logic ... const
heartbeatInterval = setInterval(() => { if (socket.data.userId &&
socket.data.sessionId) { heartbeat(socket.data.userId,
```

```
socket.data.sessionId);    }    }, 5 * 60 * 1000);    // every 5 minutes
socket.on('disconnect', () => {    clearInterval(heartbeatInterval);
trackEvent({    userId:    socket.data.userId,    sessionId:
socket.data.sessionId,    eventType: 'LOGOUT',    });    }); });
```

Important — Import heartbeat

Ensure both

trackEvent

and

heartbeat

are imported at the top of

apps/server/src/socket/index.ts

:

```
import { trackEvent, heartbeat } from '../services/analytics.js';
```

1.5 — Analytics PostgreSQL Query Routes LINUX

Create apps/server/src/routes/admin/analytics.ts. All routes under this file require the `ADMIN` role via the `requireAdmin` middleware imported from `../../middleware/auth.js`.

```
import { Router } from 'express'; import prisma from '../../lib/prisma.js';
import { requireAdmin } from '../../middleware/auth.js'; const router =
Router(); router.use(requireAdmin); // all analytics routes require ADMIN
role // — DAU
_____ //
Daily Active Users: users with at least one LOGIN in the last N days
router.get('/dau', async (req, res) => {    const days =
parseInt(req.query.days as string) || 30;    const result = await prisma.
$queryRaw<{ date: string; dau: number }[]>`    SELECT
DATE_TRUNC('day', "createdAt" AT TIME ZONE 'UTC') AS date,
COUNT(DISTINCT "userId") AS dau    FROM "GameEvent"    WHERE "eventType" =
'LOGIN'    AND "createdAt" > NOW() - INTERVAL '${days} days'    GROUP BY
DATE_TRUNC('day', "createdAt" AT TIME ZONE 'UTC')    ORDER BY date ASC `;
res.json(result); }); // — SESSION LENGTH
_____ // Average
session duration in minutes, per day, over last 30 days router.get('/session-
length', async (_req, res) => {    const result = await prisma.
$queryRaw<{ date: string; avg_minutes: number }[]>`    SELECT
```

```

DATE_TRUNC('day', login."createdAt") AS date,
AVG(
    EXTRACT(EPOCH FROM (logout."createdAt" - login."createdAt")) /
    60
) AS avg_minutes
FROM "GameEvent" login JOIN "GameEvent"
logout ON login."sessionId" = logout."sessionId" AND
logout."eventType" = 'LOGOUT' WHERE login."eventType" = 'LOGIN' AND
login."createdAt" > NOW() - INTERVAL '30 days' GROUP BY DATE_TRUNC('day',
login."createdAt") ORDER BY date ASC `; res.json(result); }); // —
ROOM POPULARITY

// Top 10 rooms
by unique visitors in the last 24 hours router.get('/room-popularity', async
(_req, res) => { const result = await prisma.$queryRaw<{ room_id: string;
visitors: number }[]>` SELECT "eventData"->>'roomId' AS room_id,
COUNT(DISTINCT "userId") AS visitors FROM "GameEvent" WHERE
"eventType" = 'ROOM_JOIN' AND "createdAt" > NOW() - INTERVAL '24 hours'
GROUP BY "eventData"->>'roomId' ORDER BY visitors DESC LIMIT 10 `;
res.json(result); }); // — ECONOMY HEALTH

// Coins earned
(via mini-games) vs coins spent (purchases) per day, 30 days
router.get('/economy', async (_req, res) => { const earned = await prisma.
$queryRaw<{ date: string; total_coins: number }[]>` SELECT
DATE_TRUNC('day', "createdAt") AS date, SUM(("eventData"-
>>'coins')::int) AS total_coins FROM "GameEvent" WHERE "eventType" IN
('FISH_CAUGHT', 'PIZZA_COMPLETED', 'CRAFTING_COMPLETED') AND
"createdAt" > NOW() - INTERVAL '30 days' GROUP BY DATE_TRUNC('day',
"createdAt") ORDER BY date ASC `; const spent = await prisma.
$queryRaw<{ date: string; total_spent: number }[]>` SELECT
DATE_TRUNC('day', "createdAt") AS date, COUNT(*) * 100 AS total_spent
-- approximate: each purchase ~100 coins FROM "GameEvent" WHERE
"eventType" = 'ITEM_PURCHASED' AND "createdAt" > NOW() - INTERVAL '30
days' GROUP BY DATE_TRUNC('day', "createdAt") ORDER BY date ASC `;
res.json({ earned, spent }); }); // — MINI-GAME COMPLETION RATES

// Event counts for each mini-
game over the last 7 days router.get('/minigames', async (_req, res) =>
{ const result = await prisma.$queryRaw<{ event_type: string; count: number
}[]>` SELECT "eventType", COUNT(*) AS count FROM "GameEvent"
WHERE "eventType" IN ('FISH_CAUGHT', 'PIZZA_COMPLETED', 'CRAFTING_COMPLETED')
AND "createdAt" > NOW() - INTERVAL '7 days' GROUP BY "eventType"
ORDER BY count DESC `; res.json(result); }); export default router;

```

Register all analytics routes in apps/server/src/routes/admin/index.ts:

```

import analyticsRoutes from './analytics.js'; // ... after existing admin
route registrations: adminRouter.use('/analytics', analyticsRoutes);

```

1.6 — Analytics Dashboard UI (Admin Page) MAC

Install Chart.js as a client dependency:

```
# [MAC] – from monorepo root pnpm add chart.js --filter client
```

Create apps/client/src/pages/admin/Analytics.ts:

```

import { Chart } from 'chart.js/auto'; export class AnalyticsDashboard
{ private container: HTMLElement; private apiBase: string;
constructor(container: HTMLElement, apiBase: string) { this.container =
container; this.apiBase = apiBase; this.render(); } private
async render() { this.container.innerHTML = ` <div
style="background:#0f1b2d;color:#e2e8f0;padding:24px;font-family:system-ui;">
<h1 style="color:#00d4c8;margin-bottom:24px;"> HavenWorld Analytics</h1>

```

```

<div style="display:grid;grid-template-columns:1fr 1fr;gap:24px;">
<div style="background:#1a2e45;border-radius:12px;padding:20px;">
<h2 style="color:#00d4c8;font-size:1rem;">Daily Active Users (30 days)</h2>
<canvas id="dau-chart" height="200"></canvas> </div> <div
style="background:#1a2e45;border-radius:12px;padding:20px;"> <h2
style="color:#00d4c8;font-size:1rem;">Avg Session Length (minutes)</h2>
<canvas id="session-chart" height="200"></canvas> </div>
<div style="background:#1a2e45;border-radius:12px;padding:20px;">
<h2 style="color:#00d4c8;font-size:1rem;">Top Rooms Today (unique
visitors)</h2> <canvas id="room-chart" height="200"></canvas>
</div> <div style="background:#1a2e45;border-
radius:12px;padding:20px;"> <h2 style="color:#00d4c8;font-
size:1rem;">Economy: Coins Earned vs Spent</h2> <canvas
id="economy-chart" height="200"></canvas> </div> </div>
</div> `; const [dauData, sessionData, roomData, economyData] =
await Promise.all([
fetch(`${this.apiBase}/api/admin/analytics/dau`).then(r => r.json()),
fetch(`${this.apiBase}/api/admin/analytics/session-length`).then(r =>
r.json()), fetch(`${this.apiBase}/api/admin/analytics/room-
popularity`).then(r => r.json()),
fetch(`${this.apiBase}/api/admin/analytics/economy`).then(r => r.json()),
]); // — DAU Chart

new
Chart(document.getElementById('dau-chart') as HTMLCanvasElement,
{ type: 'line', data: { labels: dauData.map((d: any) =>
d.date.substring(0, 10)), datasets: [{ label: 'DAU',
data: dauData.map((d: any) => Number(d.dau)), borderColor:
'#00d4c8', backgroundColor: 'rgba(0,212,200,0.1)', fill:
true, }], options: { responsive: true,
plugins: { legend: { labels: { color: '#e2e8f0' } } }, scales: {
x: { ticks: { color: '#aaa' } }, y: { ticks: { color: '#aaa' } },
beginAtZero: true }, }, }); // — Session Length
Chart

new
Chart(document.getElementById('session-chart') as HTMLCanvasElement,
{ type: 'bar', data: { labels: sessionData.map((d: any)
=> d.date.substring(0, 10)), datasets: [{ label: 'Avg
Minutes', data: sessionData.map((d: any) =>
Number(d.avg_minutes).toFixed(1)), backgroundColor:
'rgba(0,212,200,0.6)', borderColor: '#00d4c8',
borderWidth: 1, }, ], options: { responsive:
true, plugins: { legend: { labels: { color: '#e2e8f0' } } },
scales: { x: { ticks: { color: '#aaa' } }, y: { ticks:
{ color: '#aaa' }, beginAtZero: true }, }, }, }); //
— Room Popularity Chart

new Chart(document.getElementById('room-chart') as HTMLCanvasElement, {
type: 'bar', data: { labels: roomData.map((d: any) =>
d.room_id), datasets: [{ label: 'Unique Visitors',
data: roomData.map((d: any) => Number(d.visitors)),
backgroundColor: 'rgba(13,115,119,0.7)', borderColor: '#0d7377',
borderWidth: 1, }, ], options: { indexAxis: 'y',
responsive: true, plugins: { legend: { labels: { color: '#e2e8f0' } } }
}, scales: { x: { ticks: { color: '#aaa' }, beginAtZero:
true }, y: { ticks: { color:
'#aaa' } }, }, }, }); // — Economy Chart

new
Chart(document.getElementById('economy-chart') as HTMLCanvasElement,
{ type: 'line', data: { labels:
economyData.earned.map((d: any) => d.date.substring(0, 10)),
datasets: [ { label: 'Coins Earned', data:
economyData.earned.map((d: any) => Number(d.total_coins)),

```

```
borderColor: '#4ade80',                backgroundColor: 'rgba(74,222,128,0.1)',
fill: true,                            { label: 'Coins Spent',
data: economyData.spent.map((d: any) => Number(d.total_spent)),
borderColor: '#f87171',                backgroundColor: 'rgba(248,113,113,0.1)',
fill: true,                            ],
options:
{ responsive: true,                    plugins: { legend: { labels: { color:
'#e2e8f0' } } },                      scales: { x: { ticks: { color: '#aaa' } } },
y: { ticks: { color: '#aaa' }, beginAtZero:
true },                                },
}); }
```

Route Protection

Register

/admin/analytics

as a Vite client-side route guarded by an admin role check. On load, verify the JWT role claim is

ADMIN

. If not, redirect to

/

. Never rely solely on hiding the link — always check on the component mount.

§2 Player Onboarding Flow MAC + LINUX

2.1 — Why Onboarding is Critical

Research into social virtual world retention consistently finds that players who do not complete an interactive tutorial in their first session churn at dramatically higher rates within the first 24 hours. A well-designed onboarding flow for HavenWorld accomplishes three things simultaneously:

- **Mechanical literacy:** Teaches the core loop — move, chat, fish, customize — in a zero-stakes environment before the player enters any shared social space.
- **Emotional anchoring:** Creates a sense of earned belonging through small, rewarded achievements before the player has even met another user.
- **Retention signal:** Tutorial completion rate is the single most reliable leading indicator of 30-day retention for social games. Tracking it from day one gives HavenWorld an actionable metric to optimize.

2.2 — Tutorial Room Design LINUX

The tutorial runs in a dedicated database room called **Welcome Island**. Critically, this is not a shared room — every first-time player gets their own private instance generated from the tutorial template. This means new players can't be bothered, grieved, or confused by other players' avatars during onboarding.

SSH into the Oracle VM and run the following in psql:

```
# [LINUX] – SSH into Oracle VM, then: psql -U havenworld -d havenworld --
Insert the tutorial room template: INSERT INTO "Room" (name, "templateId",
"isPublic", "maxCapacity", description) VALUES ( 'Welcome Island',
'tutorial', FALSE, -- not visible in the public room list 1,
-- solo room: one player per instance 'Your first steps in HavenWorld.
Complete your orientation to earn your Welcome Badge!' );
```

Add two new Boolean fields to the Room model in

packages/shared/prisma/schema.prisma:

```
model Room { // ... existing fields ... isTutorial Boolean @default(false)
isSolo      Boolean @default(false) // true = each player gets their own
private copy }
```

In apps/server/src/socket/RoomManager.ts, add solo-room handling. When a player requests to join a room where `isSolo === true`, generate a new temporary room instance instead of joining the shared one:

```
// In RoomManager.joinRoom(): if (room.isSolo) { const tempRoomId =
`tutorial_${socket.data.userId}`; // Clone room state in Redis for this
player's private instance await redis.set(`room:${tempRoomId}`,
JSON.stringify({ ...roomTemplate, id: tempRoomId, playerCount: 1,
}), 'EX', 3600); // expire after 1 hour of inactivity return tempRoomId;
}
```

2.3 — First-Login Detection LINUX

Add tutorial progress fields to the User model in `schema.prisma`:

```
model User { // ... existing fields ... tutorialCompleted Boolean
@default(false) tutorialStep Int @default(0) // 0-5, tracks
progress through TUTORIAL_STEPS welcomeBadgeAt
DateTime? // timestamp when tutorial was finished }
# [LINUX] – on Oracle VM after pushing schema changes: npx prisma migrate
deploy
```

In `apps/server/src/socket/index.ts`, after authenticating the socket connection, handle the `PLAYER_JOIN_WORLD` event:

```
socket.on('PLAYER_JOIN_WORLD', async () => { const user = await
prisma.user.findUnique({ where: { id: socket.data.userId } }); if (!user)
return; if (!user.tutorialCompleted) { // First login – route to
private tutorial instance socket.emit('ROUTE_TO_TUTORIAL',
{ roomId: `tutorial_${user.id}`, message: "Welcome to
HavenWorld! Let's get you started.", }); trackEvent({ userId:
user.id, sessionId: socket.data.sessionId, eventType:
'TUTORIAL_STEP_COMPLETE', eventData: { step: 0 }, }); } else {
// Returning player – route to Haven Park (main social hub)
socket.emit('ROUTE_TO_ROOM', { roomId: 'haven_park' }); } });
```

2.4 — Quest Chain: "Your First Steps" MAC + LINUX

The tutorial consists of 5 sequential steps. Each step listens for a specific `Socket.io` event trigger. When the trigger fires and conditions are met, the server advances the player's `tutorialStep`, grants a reward, and emits a confirmation event back to the client.

Create `packages/shared/src/tutorial.ts`:

```
export const TUTORIAL_STEPS = [ { step: 1, id: 'first_steps',
title: '🐾 Take Your First Steps', description: 'Use WASD or arrow keys
to walk around Welcome Island.', trigger: 'PLAYER_MOVED', reward:
{ coins: 25, message: "Great! You're moving around HavenWorld!" }, }, {
step: 2, id: 'customize_avatar', title: '✨ Dress Up Your Avatar',
description: 'Open the wardrobe and change one item on your avatar.',
trigger: 'AVATAR_CUSTOMIZED', reward: { coins: 50, message: 'Looking
great! Your HavenWorld look is unique to you.' }, }, { step: 3,
id: 'catch_a_fish', title: '🎣 Try the Fishing Pond', description:
'Visit the pond and catch your first fish.', trigger: 'FISH_CAUGHT',
reward: { coins: 75, item: 'fishing_rod_basic', message:
"You're a natural angler!", }, }, { step: 4, id:
'visit_haven_park', title: '🌍 Visit Haven Park', description: 'Leave
Welcome Island and explore the main social hub.', trigger: 'ROOM_JOIN',
triggerCondition: { roomId: 'haven_park' }, reward: { coins: 100,
message: "Welcome to Haven Park! This is where all the action
happens.", }, }, { step: 5, id: 'send_a_chat', title: '💬
Say Hello to Haven!', description: 'Type anything in the chat box and
press Enter.', trigger: 'CHAT_MESSAGE', reward: { coins: 50,
badge: 'first_words', message: "You're officially a HavenWorld
```

```
citizen!",      },      }, ] as const; export type TutorialStepId = typeof
TUTORIAL_STEPS[number]['id'];
```

Create apps/server/src/socket/handlers/tutorial.ts:

```
import { TUTORIAL_STEPS } from 'shared/tutorial.js'; import prisma from
'../../lib/prisma.js'; import { trackEvent } from
'../../services/analytics.js'; import type { AuthSocket } from '../types.js';
import type { Server as SocketIOServer } from 'socket.io'; export async
function handleTutorialProgress( io: SocketIOServer, socket: AuthSocket,
trigger: string, triggerData?: Record<string, unknown> ) { const user =
await prisma.user.findUnique({ where: { id: socket.data.userId },
select: { tutorialStep: true, tutorialCompleted: true }, }); if (!user ||
user.tutorialCompleted) return; const nextStep =
TUTORIAL_STEPS[user.tutorialStep]; if (!nextStep || nextStep.trigger !==
trigger) return; // Check trigger condition (e.g., must join 'haven_park'
specifically) if ('triggerCondition' in nextStep &&
nextStep.triggerCondition) { const conditionMet =
Object.entries(nextStep.triggerCondition) .every(([k, v]) =>
triggerData?.[k] === v); if (!conditionMet) return; } const newStep
= user.tutorialStep + 1; const finished = newStep >= TUTORIAL_STEPS.length;
// Persist progress await prisma.user.update({ where: { id:
socket.data.userId }, data: { tutorialStep: newStep,
tutorialCompleted: finished, welcomeBadgeAt: finished ? new Date() :
undefined, }, }); // Grant coin reward const reward =
nextStep.reward; if (reward.coins) { await
prisma.user.update({ where: { id: socket.data.userId }, data:
{ havenCoins: { increment: reward.coins }, }, }); // Grant badge
reward if ('badge' in reward && reward.badge) { await
prisma.achievement.create({ data: { userId: socket.data.userId, type:
reward.badge, earnedAt: new Date() }, }); // Grant item reward if
('item' in reward && reward.item) { const item = await
prisma.item.findUnique({ where: { slug: reward.item } }); if (item) {
await prisma.inventory.create({ data: { userId: socket.data.userId,
itemId: item.id }, }); // Notify client of step completion
socket.emit('TUTORIAL_STEP_COMPLETED', { step: newStep, reward,
message: reward.message, finished, }); if (finished)
{ socket.emit('TUTORIAL_FINISHED', { message: '🎉 Tutorial
complete! Welcome to HavenWorld! You\'ve earned your Welcome Badge.',
badge: 'welcome_badge', totalCoinsEarned: 300, }); trackEvent({
userId: socket.data.userId, sessionId: socket.data.sessionId,
eventType: 'TUTORIAL_FINISHED', }); } else
{ trackEvent({ userId: socket.data.userId, sessionId:
socket.data.sessionId, eventType: 'TUTORIAL_STEP_COMPLETE',
eventData: { step: newStep }, }); } }
```

Wiring handleTutorialProgress

Call

handleTutorialProgress(io, socket, trigger, data)

at the end of each relevant socket event handler. For example, in

fish.ts

, after awarding coins, call

```
await handleTutorialProgress(io, socket, 'FISH_CAUGHT')
```

. In

RoomManager.ts

, after emitting

ROOM_JOIN

to the client, call

```
await handleTutorialProgress(io, socket, 'ROOM_JOIN', { roomId })
```

.

2.5 — Tutorial HUD (Client Side) MAC

Create `apps/client/src/ui/TutorialHUD.ts`:

```
import { TUTORIAL_STEPS } from 'shared/tutorial.js'; export class
TutorialHUD { private container: HTMLElement; constructor()
{ this.container = document.createElement('div');
Object.assign(this.container.style, { position: 'fixed',
bottom: '120px', right: '24px', background: '#1a2e45',
border: '2px solid #00d4c8', borderRadius: '12px', padding:
'16px 20px', zIndex: '150', color: '#e2e8f0',
maxWidth: '280px', fontFamily: 'system-ui', display: 'none',
}); document.body.appendChild(this.container); } showStep( step:
typeof TUTORIAL_STEPS[number], stepNumber: number, totalSteps: number
) { const pct = (stepNumber / totalSteps) * 100;
this.container.innerHTML = `<div style="color:#00d4c8;font-
size:0.75rem;margin-bottom:4px;"> Tutorial Step ${stepNumber}/${$
{totalSteps}</div> <div style="font-weight:bold;margin-
bottom:6px;">${step.title}</div> <div style="color:#aaa;font-
size:0.875rem;">${step.description}</div> <div style="margin-
top:12px;background:#0f1b2d;height:4px;border-radius:2px;"> <div
style="background:#00d4c8;height:100%;width:${pct}%> </div> `;
border-radius:2px;transition:width 0.5s ease;"></div> `;
this.container.style.display = 'block'; } showReward(message: string,
coins: number) { const toast = document.createElement('div');
Object.assign(toast.style, { position: 'fixed', top:
'24px', left: '50%', transform: 'translateX(-50%)',
background: '#00d4c8', color: '#0f1b2d', padding: '12px
24px', borderRadius: '30px', fontWeight: 'bold', zIndex:
```

```
'200',      fontSize: '1rem',      });      toast.textContent = `${message}
+${coins} 🏠`;      document.body.appendChild(toast);      setTimeout(() =>
toast.remove(), 3500);      }      hide() { this.container.style.display = 'none';
} } // — Wire socket events in apps/client/src/main.ts
// const tutorialHUD = new TutorialHUD(); // //
socket.on('ROUTE_TO_TUTORIAL', ({ roomId, message }) => { //
tutorialHUD.showStep(TUTORIAL_STEPS[0], 1,
TUTORIAL_STEPS.length); // }); // // socket.on('TUTORIAL_STEP_COMPLETED',
({ step, reward, message }) => { //      const stepData =
TUTORIAL_STEPS[step]; //      if (stepData) tutorialHUD.showStep(stepData, step,
TUTORIAL_STEPS.length); //      if (reward.coins)
tutorialHUD.showReward(message, reward.coins); // }); // //
socket.on('TUTORIAL_FINISHED', ({ message }) => { //      tutorialHUD.hide(); //
showCelebrationModal(message); // full-screen confetti modal // });
```

2.6 — Invite Code Redemption UX Enhancement MAC + LINUX

Alpha testers who redeem an invite code during beta still receive the **Founding Member** badge even after the beta gate fully opens. This is a persistent reward for early supporters.

LINUX When an invite code is redeemed in apps/server/src/routes/auth.ts (or wherever redemption is handled), grant the badge immediately:

```
// After marking invite code as redeemed and creating the user: await
prisma.achievement.create({ data: {      userId: newUser.id,      type:
'founding_member',      earnedAt: new Date(),      }, });
```

MAC On the client, after the registration response is received, check for the `founding_member` achievement in the returned user object and show a special welcome card:

```
// In apps/client/src/pages/Register.ts — after successful API response: if
(userData.achievements?.includes('founding_member'))
{      showFoundingMemberCard();      // animated gold card with badge description
}      function showFoundingMemberCard() {      const card =
document.createElement('div');      Object.assign(card.style, {      position:
'fixed', inset: '0',      background: 'rgba(0,0,0,0.8)',      zIndex: '500',
display: 'flex', alignItems: 'center', justifyContent: 'center',      });
card.innerHTML = `      <div style="background:linear-
gradient(135deg,#1a2e45,#0d3b5e);      border:2px solid
#f59e0b;border-radius:16px;      padding:32px;max-
width:400px;text-align:center;color:#e2e8f0;">      <div style="font-
size:3rem;margin-bottom:12px;">👑</div>      <h2
style="color:#f59e0b;margin:0 0 8px;">Welcome, Founding Member!</h2>      <p
style="color:#cbd5e1;margin-bottom:20px;">You're part of HavenWorld
history. Your golden crown badge marks you as one      of the first people
to ever set foot on these shores.      </p>      <button
onclick="this.closest('div').parentElement.remove()"
style="background:#f59e0b;color:#1a1a1a;padding:10px 28px;
border:none;border-radius:8px;font-weight:bold;cursor:pointer;">
Enter HavenWorld 🌟      </button>      </div>      `;
document.body.appendChild(card); }
```

Passport Card Crown Icon

In the player profile/passport component, query achievements and render a gold



emoji badge next to the username if

`founding_member`

is present. Keep it subtle — a small inline icon, not a banner.

§3 Live Monitoring Dashboard LINUX MAC

3.1 — Overview

With beta players active around the clock, HavenWorld's Oracle VM must be continuously observed. The monitoring stack is assembled entirely from free-tier services already in use or trivially added:

Tool	What It Monitors	Alert Channel	Cost
UptimeRobot	External HTTP availability of game, API, and assets	Email + Discord webhook	Free
PM2	Process CPU, memory, restart count	Admin dashboard	Free (already in use)
Sentry	Runtime errors, release regression tracking	Email + Sentry alerts	Free tier
/health endpoint	DB connection, Redis connection, socket count,	UptimeRobot polls this	Free (self-hosted)

Tool	What It Monitors	Alert Channel	Cost
	memory		
Discord alert.ts	Critical server errors, DB/Redis failures	Discord #server-status	Free

3.2 — Enhanced Health Endpoint with Socket Metrics LINUX

Update `apps/server/src/index.ts` to extend the `/health` endpoint created in Part 9A. The new version adds `socketCount`, full memory breakdown, and Redis status:

```
app.get('/health', async (_req, res) => { try { // Verify both critical
dependencies are alive await prisma.$queryRaw`SELECT 1`; const
redisAlive = (await redis.ping()) === 'PONG'; const socketCount =
io.engine.clientsCount; const mem = process.memoryUsage();
res.json({ status: 'ok', timestamp: new
Date().toISOString(), uptime: Math.floor(process.uptime()),
pid: process.pid, socketCount, memory:
{ heapUsed: Math.round(mem.heapUsed / 1024 / 1024) + 'MB',
heapTotal: Math.round(mem.heapTotal / 1024 / 1024) + 'MB', rss:
Math.round(mem.rss / 1024 / 1024) + 'MB', }, database:
'connected', redis: redisAlive ? 'connected' : 'error', }); }
catch (err) { res.status(503).json({ status: 'error', message:
String(err) }); } });
```

UptimeRobot Keyword Check

Set UptimeRobot to check for the keyword

"status":"ok"

in the HTTP response body in addition to the HTTP 200 status code. This way, a 200 response that contains

"status":"error"

(e.g., if DB is down but Express is still running) will correctly trigger a downtime alert.

3.3 — UptimeRobot Public Status Page LINUX

Configure UptimeRobot to create a public-facing status page for players and community members:

Step 1: In the UptimeRobot dashboard, navigate to **Status Pages → Add Status Page**.

Step 2: Name it **HavenWorld Status**. Add the following three monitors to the page:

Monitor Name	URL / Endpoint	Check Interval	Check Type
HavenWorld Game	havenworld.pages.dev	Every 5 minutes	HTTP(S)
HavenWorld API	[oracle-ip].nip.io/health	Every 5 minutes	HTTP(S) + Keyword: "status": "ok"
HavenWorld Assets	assets.havenworld.pages.dev	Every 5 minutes	HTTP(S)

Step 3: Enable **Subscriber Notifications** on the status page — players can opt in to receive email alerts about outages.

Step 4: Copy the public status page URL (e.g., `https://stats.uptimerobot.com/[your-id]`). Add it to:

- README.md — under a "Live Status" badge at the top
- Discord server description — "Live status: [url]"
- HavenWorld Hashnode blog About section

Configure alerting thresholds: alert only when downtime exceeds 2 continuous minutes, to avoid false positives from transient network blips. Alert destinations:

- Email: `dboone9@msn.com`
- Discord webhook: Discord server → **#server-status** channel → Edit Channel → Integrations → New Webhook → Copy URL → paste into UptimeRobot Alert Contacts

3.4 — Sentry Release Tracking MAC

With Sentry already integrated from Part 8, add release tracking to both deploy workflows so Sentry knows exactly which Git commit is running in production:

Update `.github/workflows/deploy-server.yml` — add after the SSH deploy step:

```
- name: Create Sentry Release (Server)  uses: getsentry/action-release@v1
env:      SENTRY_AUTH_TOKEN: ${ secrets.SENTRY_AUTH_TOKEN }      SENTRY_ORG:
${ secrets.SENTRY_ORG }      SENTRY_PROJECT:  havenworld-server  with:
environment: production      version:      ${ github.sha }      set_commits:
auto
```

Update `.github/workflows/deploy-client.yml` — add after the Cloudflare Pages deploy step:

```
- name: Create Sentry Release (Client)  uses: getsentry/action-release@v1
env:      SENTRY_AUTH_TOKEN: ${ secrets.SENTRY_AUTH_TOKEN }      SENTRY_ORG:
${ secrets.SENTRY_ORG }      SENTRY_PROJECT:  havenworld-client  with:
environment: production      version:      ${ github.sha }      set_commits:
auto
```

Add to GitHub Secrets (Settings → Secrets → Actions → New repository secret):

- `SENTRY_AUTH_TOKEN` — from Sentry dashboard → Settings → Auth Tokens → Create New Token (scopes: `project:releases`, `org:read`)
- `SENTRY_ORG` — your Sentry organization slug (visible in the Sentry URL)

What This Unlocks

Once release tracking is active: (1) each Sentry error shows the exact GitHub commit that introduced it; (2) you can mark issues as "Resolved in version X" and Sentry automatically re-opens them if they regress in a later release; (3) the Sentry dashboard shows per-release error rates, so you can immediately see if a deploy made stability better or worse.

3.5 — PM2 Metrics Admin Route LINUX

Create `apps/server/src/routes/admin/metrics.ts` to expose PM2 process metrics via a protected API endpoint. The admin analytics dashboard will poll this to display a "Server Health" panel alongside the game analytics charts.

```
import { Router } from 'express'; import { execSync } from 'child_process';
import { requireAdmin } from '../middleware/auth.js'; const router =
Router(); router.use(requireAdmin); router.get('/pm2', (_req, res) =>
{ try { const raw = execSync('pm2 jlist', { encoding: 'utf8' });
const processes = JSON.parse(raw); const summary = processes.map((p:
any) => ({ name: p.name, pid: p.pid, status:
p.pm2_env.status, cpu: p.monit?.cpu, memoryMB:
p.monit?.memory ? Math.round(p.monit.memory / 1024 / 1024)
: null, uptime: p.pm2_env.pm_uptime, restarts:
p.pm2_env.restart_time, })); res.json(summary); } catch (err) {
res.status(500).json({ error: String(err) }); } }); export default router;
```

Register in `apps/server/src/routes/admin/index.ts`:

```
import metricsRoutes from './metrics.js'; adminRouter.use('/metrics',
metricsRoutes);
```

In the `AnalyticsDashboard` component (from §1.6), add a server health table below the four charts:

```
const pm2Data = await fetch(`${this.apiBase}/api/admin/metrics/pm2`).then(r
=> r.json()); const healthTable = document.createElement('div');
healthTable.style.cssText = 'background:#1a2e45;border-
radius:12px;padding:20px;margin-top:24px;'; healthTable.innerHTML = ` <h2
style="color:#00d4c8;font-size:1rem;margin-top:0;">🖥️ Server Health
(PM2)</h2> <table style="width:100%;border-collapse:collapse;font-
size:0.875rem;"> <thead> <tr style="color:#00d4c8;"> <th
style="text-align:left;padding:6px 8px;border-bottom:1px solid
#2d4a6e;">Instance</th> <th style="text-align:left;padding:6px
8px;border-bottom:1px solid #2d4a6e;">Status</th> <th style="text-
align:left;padding:6px 8px;border-bottom:1px solid #2d4a6e;">CPU %</th>
<th style="text-align:left;padding:6px 8px;border-bottom:1px solid
#2d4a6e;">RAM (MB)</th> <th style="text-align:left;padding:6px
8px;border-bottom:1px solid #2d4a6e;">Restarts</th> </tr> </thead>
<tbody> ${pm2Data.map((p: any) => ` <tr style="color:#e2e8f0;">
<td style="padding:6px 8px;">${p.name}</td> <td style="padding:6px
8px;color:${p.status === 'online' ? '#4ade80' : '#f87171'};">
${p.status}</td> <td style="padding:6px 8px;">${p.cpu ??
'-'}%</td> <td style="padding:6px 8px;">${p.memoryMB ?? '-'}</td>
<td style="padding:6px 8px;">${p.restarts}</td> </tr>
`).join('')} </tbody> </table> `;
this.container.querySelector('div')!.appendChild(healthTable);
```

3.6 — Discord Incident Webhook LINUX

Create `apps/server/src/services/alert.ts` for automated incident notifications sent directly to your Discord **#server-status** channel:

```
const DISCORD_WEBHOOK = process.env.DISCORD_ALERT_WEBHOOK; export type
AlertLevel = 'info' | 'warn' | 'error'; export async function
```

```

sendAlert(  title:  string,  message: string,  level:  AlertLevel =
'info' ): Promise<void> {  if (!DISCORD_WEBHOOK) return;  const colors:
Record<AlertLevel, number> = {    info:  0x00d4c8,    // teal    warn:
0xf59e0b,    // amber    error: 0xef4444,    // red  };  try {    await
fetch(DISCORD_WEBHOOK, {      method: 'POST',      headers: { 'Content-
Type': 'application/json' },      body: JSON.stringify({        embeds: [{
title,        description: message,        color:        colors[level],
timestamp:    new Date().toISOString(),        footer:        { text:
'HavenWorld Auto-Alert' },        }],        })),    });  } catch (err) {
console.error('[Alert] Failed to send Discord webhook:', err);  } }

```

Call `sendAlert` from your Prisma and Redis connection error handlers:

```

// In apps/server/src/lib/prisma.ts – in the $on('error') handler: prisma.
$on('error', (e) => {  sendAlert('🔴 Database Error', e.message,
'error'); }); // In apps/server/src/lib/redis.ts – in the 'error' event
listener: redis.on('error', (err) => {  sendAlert('🟡 Redis Disconnected',
String(err), 'warn'); });

```

Add `DISCORD_ALERT_WEBHOOK` to the Oracle VM `.env` file and to GitHub Secrets:

```

# [LINUX] – on Oracle VM, append to /home/ubuntu/havenworld/.env: echo
"DISCORD_ALERT_WEBHOOK=https://discord.com/api/webhooks/[your-webhook-url]" \
>> /home/ubuntu/havenworld/.env # Reload PM2 to pick up the new env
variable: pm2 restart havenworld-server --update-env

```

§4 Seasonal Event Activation System MAC + LINUX

4.1 — How the System Works

HavenWorld's seasonal events — Winter Festival, Spring Fair, Summer Beach Bash, and Autumn Harvest — activate and deactivate **without any server restart or deploy**. The mechanism is deliberately simple and resilient:

- A single Redis key stores the active season: `season:active = "winter"` (or `"none"`)
- The server reads this key on each relevant request with a lightweight Redis `GET`
- An admin API endpoint lets Daniel activate or deactivate a season from any browser

- The activation endpoint immediately `io.emit()`s `SEASON_CHANGED` to all connected clients — the world theme updates in real time with zero page reload
- The client reads the active season on connect and applies the theme to the Babylon.js scene

Design Rationale — Redis over Database

A database row would require an additional Prisma query on every fish catch, catalog load, and room join. Redis

GET

is sub-millisecond and requires no connection pool slot. For a value that changes at most a few times per year, Redis is the correct choice.

4.2 — Season Definitions MAC

Create `packages/shared/src/seasons.ts`. This file is the single source of truth for all season visual properties, catalog item slugs, and fish coin bonuses. Both the server and the client import from this shared package.

```
export type SeasonId = 'none' | 'winter' | 'spring' | 'summer' | 'autumn';
export const SEASONS = {
  none: {
    id: 'none',
    name: 'Standard',
    theme: {
      skyColor: '#87CEEB',
      ambientColor: '#ffffff',
      fogDensity: 0.01,
      catalogItems: [] as string[],
      fishBonus: 0,
    },
  },
  winter: {
    id: 'winter',
    name: 'Winter Festival ❄️',
    theme: {
      skyColor: '#c8d8e8',
      ambientColor: '#aac8ff',
      fogDensity: 0.03,
      catalogItems: ['ice_throne', 'snowflake_lamp', 'fireplace_cozy', 'elf_hat', 'santa_coat'],
      fishBonus: 0.20, // +20% coin bonus on fish caught during winter
      roomOverlays: ['snowfall_particle_system'],
    },
  },
  spring: {
    id: 'spring',
    name: 'Spring Fair 🌸',
    theme: {
      skyColor: '#ffe4e1',
      ambientColor: '#ffcce0',
      fogDensity: 0.008,
      catalogItems: ['flower_arch', 'cherry_blossom_tree', 'bunny_hat', 'spring_dress'],
      fishBonus: 0.10, // +10% coin bonus
      roomOverlays: ['petal_particle_system'],
    },
  },
  summer: {
    id: 'summer',
    name: 'Summer Beach Bash 🏖️',
    theme: {
      skyColor: '#00bfff',
      ambientColor: '#fff0a0',
      fogDensity: 0.005,
      catalogItems: ['beach_umbrella', 'surfboard_sofa', 'flamingo_float', 'sunhat'],
      fishBonus: 0.15, // +15% coin bonus
      roomOverlays: ['sparkle_particle_system'],
    },
  },
  autumn: {
    id: 'autumn',
    name:

```

```
'Autumn Harvest 🍂', theme: { skyColor: '#d2691e',
ambientColor: '#ff8c00', fogDensity: 0.02, }, catalogItems:
['pumpkin_throne', 'autumn_wreath', 'scarecrow_lamp', 'harvest_table'],
fishBonus: 0.25, // +25% – autumn fish migration! highest
seasonal bonus roomOverlays: ['leaf_particle_system'], }, } as const;
export type Season = typeof SEASONS[SeasonId];
```

4.3 — Season Service (Server) LINUX

Create `apps/server/src/services/season.ts`:

```
import redis from '../lib/redis.js'; import { SEASONS, type SeasonId, type
Season } from 'shared/seasons.js'; const SEASON_KEY = 'season:active';
export async function getActiveSeason(): Promise<Season> { const seasonId =
(await redis.get(SEASON_KEY) as SeasonId) ?? 'none'; return
(SEASONS[seasonId] ?? SEASONS.none) as Season; } export async function
setActiveSeason(seasonId: SeasonId): Promise<void> { await
redis.set(SEASON_KEY, seasonId); // No expiry – season stays active until
manually changed by admin } export async function clearSeason():
Promise<void> { await redis.set(SEASON_KEY, 'none'); }
```

4.4 — Season Admin API LINUX

Create `apps/server/src/routes/admin/season.ts`:

```
import { Router } from 'express'; import { getActiveSeason, setActiveSeason,
clearSeason } from '../services/season.js'; import { requireAdmin } from
'../middleware/auth.js'; import { io } from '../index.js';
// exported Socket.io server instance import type { SeasonId } from
'shared/seasons.js'; const router = Router(); router.use(requireAdmin); //
GET /api/admin/season – current active season router.get('/', async (_req,
res) => { const season = await getActiveSeason();
res.json(season); }); // POST /api/admin/season/activate/:seasonId –
activate a season router.post('/activate/:seasonId', async (req, res) =>
{ const { seasonId } = req.params; await setActiveSeason(seasonId as
SeasonId); const season = await getActiveSeason(); // Broadcast
immediately to ALL connected clients – no reload needed
io.emit('SEASON_CHANGED', season); res.json({ success: true, message: `
${seasonId} season activated`, season }); }); // POST
/api/admin/season/deactivate – return to standard (none)
router.post('/deactivate', async (_req, res) => { await clearSeason();
const season = await getActiveSeason(); io.emit('SEASON_CHANGED', season);
res.json({ success: true, message: 'Season cleared – returning to standard',
season }); }); export default router;
```

Register in `apps/server/src/routes/admin/index.ts`:

```
import seasonRoutes from './season.js'; adminRouter.use('/season',
seasonRoutes);
```

Also expose a **public** read-only season endpoint for the client to check on connect:

```
// In apps/server/src/index.ts – public route, no auth required:
app.get('/api/season', async (_req, res) => { const season = await
getActiveSeason(); res.json(season); });
```

4.5 — Season Client Integration MAC

In `apps/client/src/world/SceneManager.ts`, apply the seasonal theme when the socket connects and handle live season changes:

```
import * as BABYLON from '@babylonjs/core'; import { SEASONS } from
'shared/seasons.js'; import type { Season } from 'shared/seasons.js'; //
Called once on socket connect socket.on('connect', async () => { const
season: Season = await fetch(`${API_BASE}/api/season`).then(r => r.json());
applySeasonTheme(scene, season); }); // Called in real time when admin
changes the season socket.on('SEASON_CHANGED', (season: Season) =>
{ applySeasonTheme(scene, season); showSeasonToast(season.name); //
e.g., "❄️ Winter Festival has begun!" }); function applySeasonTheme(scene:
BABYLON.Scene, season: Season) { // Sky / clear color const sky =
season.theme.skyColor; scene.clearColor = new
BABYLON.Color4( parseInt(sky.slice(1, 3), 16) / 255,
parseInt(sky.slice(3, 5), 16) / 255, parseInt(sky.slice(5, 7), 16) / 255,
1, ); // Ambient light color const amb = season.theme.ambientColor;
scene.ambientColor = new BABYLON.Color3( parseInt(amb.slice(1, 3), 16) /
255, parseInt(amb.slice(3, 5), 16) / 255, parseInt(amb.slice(5, 7),
16) / 255, ); // Fog scene.fogDensity = season.theme.fogDensity;
scene.fogMode = scene.fogDensity > 0 ? BABYLON.Scene.FOGMODE_EXP :
BABYLON.Scene.FOGMODE_NONE; // Particle overlays (snowfall, petals,
sparkles, leaves) if ('roomOverlays' in season && season.roomOverlays) {
(season.roomOverlays as string[]).forEach(overlayId =>
{ loadParticleSystem(scene, overlayId); }); } } function
showSeasonToast(name: string) { const toast =
document.createElement('div'); Object.assign(toast.style, { position:
'fixed', top: '80px', left: '50%', transform: 'translateX(-50%)',
background: '#1a2e45', border: '2px solid #00d4c8', color: '#e2e8f0',
padding: '12px 28px', borderRadius: '30px', fontWeight: 'bold',
zIndex: '180', fontSize: '1rem', }); toast.textContent = `❄️ ${name} has
begun!`; document.body.appendChild(toast); setTimeout(() =>
toast.remove(), 4000); }
```

In the catalog route (`apps/server/src/routes/catalog.ts`), include season-specific items automatically when a season is active:

```
import { getActiveSeason } from '../services/season.js'; router.get('/',
async (_req, res) => { const season = await getActiveSeason(); const
baseItems = await prisma.item.findMany({ where: { isActive: true } });
const seasonSlugs = 'catalogItems' in season ? [...season.catalogItems] : [];
const seasonItems = seasonSlugs.length > 0 ? await prisma.item.findMany({
where: { slug: { in: seasonSlugs } } }) : []; // Merge, deduplicate by
ID const allItems = [ ...new Map([...baseItems, ...seasonItems].map(i
=> [i.id, i])).values() ]; res.json(allItems); });
```

4.6 — Fish Coin Bonus During Seasons LINUX

In `apps/server/src/socket/handlers/fish.ts`, apply the seasonal bonus multiplier after computing the base reward:

```
import { getActiveSeason } from '../services/season.js'; // After
computing baseReward from catch quality: const season = await
getActiveSeason(); const baseReward = calculateFishReward(catchResult);
const seasonBonus = Math.floor(baseReward * season.fishBonus); const
```

```
totalReward = baseReward + seasonBonus; if (seasonBonus > 0)
{  socket.emit('FISH_SEASON_BONUS', {    bonus:      seasonBonus,
seasonName: season.name,    }); } await awardCoins(userId, totalReward);
trackEvent({  userId, sessionId, eventType: 'FISH_CAUGHT',  eventData: {
fishId:      catchResult.fishId,    weight:      catchResult.weight,
baseReward,    seasonBonus,    totalReward,    season:
season.id,    }, });
```

Testing Seasonal Activation

Test the full seasonal loop locally:

1. Start the dev server and open the client in a browser
2. Call `POST /api/admin/season/activate/winter` from Postman or curl with your admin JWT
3. Verify the browser immediately updates sky color, fog, and shows the toast without a page reload
4. Catch a fish — verify the +20% bonus appears and is logged in the GameEvent table
5. Open the catalog — verify winter items appear alongside base items
6. Call `POST /api/admin/season/deactivate` — verify everything returns to standard

§5 Patch Notes & "What's New"

Modal MAC + LINUX

5.1 — The Hashnode Integration

Daniel publishes patch notes and dev updates on Hashnode — a free blogging platform with an RSS feed, a public API, and a clean reading experience. Rather than maintaining a separate patch notes page within HavenWorld itself, the client reads from the Hashnode RSS feed via a lightweight server-side relay. When a new

post is published, an hourly GitHub Actions workflow detects it, writes the post metadata to Redis, and every player sees an unobtrusive "What's New" modal the next time they log in — but only once per post, tracked by `localStorage`.

5.2 — Hashnode Setup MAC

Step 1: Go to **hashnode.com** and create an account using your development email address.

Step 2: Create a Blog with the subdomain: `havenworld.hashnode.dev`

Step 3: Publish at least one post before beta launch. Suggested first post title: **"HavenWorld Open Beta is LIVE! 🎉"**. This post becomes the first "What's New" modal all beta players see on their first login.

Step 4: Establish a patch note naming convention and stick to it: `HavenWorld Patch Notes – v0.2.0 (Month Day, Year)`. Consistent titles make RSS parsing reliable.

Custom Domain (Optional)

Hashnode supports custom domains. If you own

`havenworld.gg`

or another domain, you can map the blog to

`blog.havenworld.gg`

. The RSS feed URL changes accordingly. Update the GitHub Actions workflow URL if you do this.

5.3 — GitHub Action: Fetch Latest Post → Redis LINUX

Create `.github/workflows/fetch-patch-notes.yml`:

```
name: Fetch Latest Patch Notes on: schedule: - cron: '0 * * * *' #
Check Hashnode every hour, on the hour workflow_dispatch: # Allow
manual trigger from GitHub Actions tab jobs: fetch-notes: runs-on:
ubuntu-latest steps: - name: Fetch latest Hashnode post from RSS
id: fetch run: | FEED=$(curl -s
https://havenworld.hashnode.dev/rss.xml) # Extract second <title>
(first is the blog title, second is the latest post) TITLE=$(echo
"$FEED" | grep -oP '(?<=<title>)[^<]+' | sed -n '2p') LINK=$(echo
"$FEED" | grep -oP '(?<=<link>)[^<]+' | sed -n '2p') PUB=$(echo
"$FEED" | grep -oP '(?<=<pubDate>)[^<]+' | head -1) echo
"title=$TITLE" >> $GITHUB_OUTPUT echo "link=$LINK" >>
$GITHUB_OUTPUT echo "pubDate=$PUB" >> $GITHUB_OUTPUT -
name: Push to Redis on Oracle VM via SSH uses: appleboy/ssh-
action@v1.0.3 with: host: ${ secrets.ORACLE_HOST }}
username: ubuntu key: ${ secrets.ORACLE_SSH_KEY }}
envs: PATCH_TITLE,PATCH_LINK,PATCH_DATE,REDIS_PASSWORD script:
| redis-cli -a "$REDIS_PASSWORD" SET patch:title "$PATCH_TITLE"
redis-cli -a "$REDIS_PASSWORD" SET patch:link "$PATCH_LINK"
redis-cli -a "$REDIS_PASSWORD" SET patch:date "$PATCH_DATE" #
INCR only if the title changed – prevents re-showing same post
CURRENT=$(redis-cli -a "$REDIS_PASSWORD" GET patch:title) if
[ "$CURRENT" != "$PATCH_TITLE" ]; then redis-cli -a
"$REDIS_PASSWORD" INCR patch:version fi redis-cli -a
"$REDIS_PASSWORD" SET patch:title "$PATCH_TITLE" env:
PATCH_TITLE: ${ steps.fetch.outputs.title }} PATCH_LINK: $
${ steps.fetch.outputs.link }} PATCH_DATE: $
${ steps.fetch.outputs.pubDate }} REDIS_PASSWORD: $
${ secrets.REDIS_PASSWORD }}
```

Add REDIS_PASSWORD to GitHub Secrets

Go to GitHub repo → Settings → Secrets → Actions → New repository secret.

Name:

REDIS_PASSWORD

. Value: the same Redis password in the Oracle VM's

.env

file. This secret is already referenced by the deploy workflow but must be explicitly set if not yet done.

5.4 — Patch Notes API Route LINUX

Create `apps/server/src/routes/patch-notes.ts`. This is a **public** endpoint — no authentication required, as it serves unauthenticated players checking for updates:

```
import { Router } from 'express'; import redis from '../lib/redis.js'; const
router = Router(); router.get('/', async (_req, res) => { const [title,
link, date, version] = await Promise.all([ redis.get('patch:title'),
redis.get('patch:link'), redis.get('patch:date'),
redis.get('patch:version'), ]); if (!title) { return
res.json({ available: false }); } res.json({ available: true,
title, link, date, version: parseInt(version ?? '1',
10), }); }); export default router;
```

Register the route in `apps/server/src/index.ts`:

```
import patchNotesRoutes from '../routes/patch-notes.js'; app.use('/api/patch-
notes', patchNotesRoutes);
```

5.5 — "What's New" Modal (Client) MAC

Create `apps/client/src/ui/WhatsNewModal.ts`. The modal uses `localStorage` to track which patch version the player has already seen, so it only appears once per unique post — not on every login:

```
interface PatchData { available: boolean; title: string; link:
string; date: string; version: number; } export class WhatsNewModal {
private readonly STORAGE_KEY = 'havenworld_last_patch_version'; async
checkAndShow(apiBase: string): Promise<void> { try { const data:
PatchData = await fetch(`${apiBase}/api/patch-notes`).then(r => r.json());
if (!data.available) return; const lastSeen =
parseInt(localStorage.getItem(this.STORAGE_KEY) ?? '0', 10); if
(data.version <= lastSeen) return; // player already dismissed this version
this.show(data); } catch { // Silently ignore — patch notes are
non-critical } } private show(data: PatchData) { const overlay =
document.createElement('div'); Object.assign(overlay.style,
{ position: 'fixed', inset: '0', background:
'rgba(0,0,0,0.75)', zIndex: '300', display:
'flex', alignItems: 'center', justifyContent:
'center', }); const formattedDate = (() => { try { return new
Date(data.date).toLocaleDateString(); } catch { return data.date; }
})(); overlay.innerHTML = ` <div
style="background:#1a2e45;border:2px solid #00d4c8;border-radius:16px;
padding:32px;max-width:480px;width:90%;text-align:center;color:#e2e8f0;">
<div style="font-size:2rem;margin-bottom:12px;"><img alt="Patch icon" data-bbox="795 755 815 775"/></div>
<h2 style="color:#00d4c8;margin:0 0 8px;font-size:1.25rem;">${data.title}</h2>
<p style="color:#aaa;font-size:0.875rem;margin-bottom:20px;">
Published: ${formattedDate} </p>
<p style="color:#cbd5e1;margin-bottom:24px;line-height:1.5;">
HavenWorld has been updated! Read the full patch notes to see
everything that's new, fixed, and improved. </p>
<div style="display:flex;gap:12px;justify-content:center;flex-wrap:wrap;">
<a href="${data.link}" target="_blank" rel="noopener noreferrer"
style="background:#00d4c8;color:#0f1b2d;padding:10px 24px;
border-radius:8px;text-decoration:none;font-weight:bold;">
Patch Notes <img alt="External link icon" data-bbox="235 910 255 930"/> </a>
<button id="whats-new-close" style="background:#00d4c8;color:#0f1b2d;padding:10px 24px;
border-radius:8px;text-decoration:none;font-weight:bold;">Read
Patch Notes </button> </div>`;
```

```

style="background:#2d4a6e;color:#e2e8f0;padding:10px 24px;
border-radius:8px;border:none;cursor:pointer;font-size:1rem;">
Continue to HavenWorld      </button>      </div>      </div>      `;
document.body.appendChild(overlay);      document.getElementById('whats-new-
close')!.addEventListener('click', () =>
{      localStorage.setItem(this.STORAGE_KEY, String(data.version));
overlay.remove();      });      // Also dismiss and mark as seen if player
clicks "Read Patch Notes"
overlay.querySelector('a')!.addEventListener('click', () =>
{      localStorage.setItem(this.STORAGE_KEY, String(data.version));
setTimeout(() => overlay.remove(), 300);      });      } }

```

Call `checkAndShow` from `apps/client/src/main.ts` after the player successfully loads into the world (after `PLAYER_JOIN_WORLD` completes, not at login, to avoid interfering with the tutorial flow):

```

import { WhatsNewModal } from './ui/WhatsNewModal.js'; // After
PLAYER_JOIN_WORLD is confirmed and the scene is loaded:
socket.on('SCENE_READY', async () => { // Delay slightly so the world
renders first, then show modal      setTimeout(async () => {      const whatsNew
= new WhatsNewModal();      await whatsNew.checkAndShow(API_BASE);      }, 2000);
});

```

Tutorial Interaction

If the player is in tutorial mode (

`tutorialCompleted === false`

), the "What's New" modal should be suppressed. In

`main.ts`

, check

`socket.data.tutorialCompleted`

(or a local flag set when

`ROUTE_TO_TUTORIAL`

is received) before calling

`checkAndShow`

.

5.6 — Manual Patch Notes Trigger MAC + LINUX

For urgent patches that cannot wait for the next hourly cron run, trigger the workflow manually:

Step 1: Go to the GitHub repository → **Actions** tab → Find "**Fetch Latest Patch Notes**" in the left sidebar.

Step 2: Click **Run workflow** → **Run workflow** (on the `main` branch). The workflow runs within ~30 seconds and updates Redis immediately.

Step 3: All currently-connected players will see the modal on their next login. Players already in-game will see it the next time they reload or reconnect.

Step 4: To verify the Redis keys were set correctly, SSH into the Oracle VM and run:

```
# [LINUX] – verify Redis patch keys after workflow run: redis-cli -a "$REDIS_PASSWORD" MGET patch:title patch:link patch:date patch:version
```

§6 Part 9B Completion Checklist

Before moving on to Part 10, verify every item below. Check each box as you complete it. All items should be checked before HavenWorld opens to public beta.

Analytics

- `GameEvent` Prisma model added to `packages/shared/prisma/schema.prisma`
- `npx prisma migrate dev --name add_game_events` ran successfully — migration file committed to Git
- `analytics.ts trackEvent` service created — fire-and-forget, never throws, always catches

- `trackEvent` integrated into: auth login route (with `sessionId` in JWT), `RoomManager` join/leave, `fish.ts` caught handler, `trade.ts` complete handler, `economy.ts` purchase handler
- 5-minute heartbeat interval added to `socket/index.ts` connection handler — cleared on disconnect
- Admin routes created and responding correctly: `/api/admin/analytics/dau`, `/session-length`, `/room-popularity`, `/economy`, `/minigames`
- Chart.js installed: `pnpm add chart.js --filter client`
- Analytics dashboard component created at `apps/client/src/pages/admin/Analytics.ts` and route registered at `/admin/analytics`
- Dashboard displays all four charts: DAU, session length, room popularity, economy health
- PM2 metrics panel renders server health table (instance, status, CPU %, RAM, restarts)

Player Onboarding

- `tutorialCompleted`, `tutorialStep`, and `welcomeBadgeAt` fields added to `User` model in `schema.prisma`
- Migration run on Oracle VM: `npx prisma migrate deploy`
- `isTutorial` and `isSolo` Boolean fields added to `Room` model
- Welcome Island room inserted into the database with `isTutorial: true`, `isSolo: true`, `maxCapacity: 1`
- Solo room instance logic added to `RoomManager.ts` — each player gets `tutorial_[userId]` private instance
- `TUTORIAL_STEPS` array defined in `packages/shared/src/tutorial.ts` with all 5 steps
- `handleTutorialProgress` function created in `apps/server/src/socket/handlers/tutorial.ts`
- `handleTutorialProgress` called from all 5 trigger event handlers: `PLAYER_MOVED`, `AVATAR_CUSTOMIZED`, `FISH_CAUGHT`, `ROOM_JOIN` (haven_park), `CHAT_MESSAGE`
- `TutorialHUD.ts` component created and appended to DOM on game load

- Socket events wired: `ROUTE_TO_TUTORIAL`, `TUTORIAL_STEP_COMPLETED`, `TUTORIAL_FINISHED` all trigger HUD updates
- Tutorial tested end-to-end with a fresh account — all 5 steps complete, coins awarded correctly ($25 + 50 + 75 + 100 + 50 = 300$ total)
- Celebration modal shown on tutorial completion
- Founding Member achievement granted on invite code redemption
- Founding Member animated card shown after registration for invite-code users

Live Monitoring

- `/health` endpoint updated to include `socketCount`, `heapUsed`, `heapTotal`, `rss`, and Redis status
- UptimeRobot public status page created — "HavenWorld Status" — with 3 monitors configured
- UptimeRobot alerting set: 2-minute downtime threshold, email + Discord webhook
- Discord webhook URL added to UptimeRobot alert contacts
- Sentry release step added to `deploy-server.yml` GitHub Actions workflow
- Sentry release step added to `deploy-client.yml` GitHub Actions workflow
- `SENTRY_AUTH_TOKEN` added to GitHub Secrets
- `SENTRY_ORG` added to GitHub Secrets
- PM2 metrics admin route created at `apps/server/src/routes/admin/metrics.ts` — `/api/admin/metrics/pm2` returns process data
- `alert.ts` Discord webhook service created
- `sendAlert` called on Prisma error event and Redis error event
- `DISCORD_ALERT_WEBHOOK` added to Oracle VM `.env` and reloaded via `pm2 restart havenworld-server --update-env`
- Status page URL published in `README.md` and Discord server description

Seasonal Events

- `SEASONS` constant and `SeasonId` type defined in `packages/shared/src/seasons.ts`
- `season.ts` service created: `getActiveSeason`, `setActiveSeason`, `clearSeason`
- Season admin routes created — `GET /api/admin/season`, `POST /api/admin/season/activate/:seasonId`, `POST /api/admin/season/deactivate`
- Public `GET /api/season` route added for unauthenticated client reads
- `io.emit('SEASON_CHANGED')` called on every admin activate/deactivate — all clients update in real time
- `applySeasonTheme()` implemented in `SceneManager.ts` — updates sky color, ambient light, fog density
- Particle system overlays loaded per season (snowfall, petals, sparkles, leaves)
- Seasonal fish coin bonus applied in `fish.ts` — bonus amount logged in `GameEvent eventData`
- Seasonal catalog items included in `/api/catalog` response when season is active
- Manual end-to-end test passed: activate winter → sky changes in browser → fish bonus works → catalog shows winter items → deactivate → all returns to standard

Patch Notes Modal

- Hashnode blog created at `havenworld.hashnode.dev`
- First blog post published: "HavenWorld Open Beta is LIVE! 🎉"
- `.github/workflows/fetch-patch-notes.yml` created with hourly schedule + manual trigger
- `REDIS_PASSWORD` added to GitHub Secrets
- GitHub Actions workflow tested: manual trigger → runs successfully → Redis keys populated
- Verified with `redis-cli MGET patch:title patch:link patch:date patch:version` on Oracle VM
- Patch notes API route created at `/api/patch-notes` — returns `{ available, title, link, date, version }`

- `WhatsNewModal.ts` created with `localStorage` version tracking
- `checkAndShow` called from `main.ts` after scene is ready (2-second delay, suppressed during tutorial)
- Manual test: increment `patch:version` in Redis → reload client → modal appears with correct title and link
- Dismiss and re-open test: dismiss modal → reload → modal does not reappear (`localStorage` correctly prevents it)
- "Read Patch Notes" link opens Hashnode post in new tab and marks version as seen

§7 What's Next — Part 10 Preview

Part 10 is the final chapter of the HavenWorld Build Guide series. With the game live in public beta, analytics running, monitoring active, and the onboarding loop proven, Part 10 focuses on long-term sustainability and growth.

Part 10 Topic Map

Topic

Summary

Scaling Beyond Oracle Free Tier

Migration paths to Hetzner CX22 (~€4/mo), fly.io hobby tier, or Railway when the Oracle ARM VM reaches its limits. When to migrate and what changes.

Horizontal Scaling

Socket.io Redis Pub/Sub adapter (already using Redis — minimal change), PgBouncer connection pooling, Cloudflare Load Balancing for multi-VM setups.

Open-Source Community Health

CONTRIBUTING.md

, good-first-issue labeling workflow,

CODE_OF_CONDUCT.md

, volunteer moderator program with in-game mod badge.

Phase 4+ Roadmap

Mobile via Capacitor.js, WebXR / VR mode (Babylon.js WebXR support), player-owned Neighborhoods, User-Generated Furniture (GLTF upload pipeline).

Ethical Monetization

Cosmetics-only gem bundles, seasonal pass (cosmetic rewards only, never stat advantages), transparent pricing, no loot boxes, no pay-to-win — ever.

Technical Debt Roadmap

WebGPU renderer readiness (Babylon.js 8 migration path), GDPR compliance audit (data deletion endpoint, right to access export), WCAG 2.1 AA accessibility

audit.

Part 9B Complete — You're Operationally Ready

With Part 9B finished, HavenWorld now has everything needed to run a live game responsibly: a self-hosted analytics system that respects player privacy, an onboarding flow proven to improve 30-day retention, 24/7 monitoring with automated alerting, a zero-deploy seasonal event system, and a frictionless patch notes delivery pipeline. The stack remains 100% free-tier. The codebase remains GPL-3.0 open source. HavenWorld is ready for public beta.

HavenWorld Build Guide — Part 9B • Prepared by Daniel • Friday, 18 September 2026, 17:30 CDT • Anoka, MN • GPL-3.0 Open Source